

ADR-002 — Elección de base de datos principal para la capa de persistencia del panel

ID	Título	Estado	Fecha
ADR-002	Elección de base de datos principal para la capa de persistencia del panel	Aceptada	27/05/2026

1. Contexto

El panel de Polizing (`my-app`) gestiona el ciclo de vida completo de productos de seguros: clientes (corporativos y personas), aseguradoras, pólizas, siniestros con documentos adjuntos, y pagos que agrupan varias pólizas en un mismo batch. El modelo de datos quedó documentado en `Entregables/Entrega 3/modelo-datos.md` (10 tablas, relaciones 1:N reales sin tablas puente innecesarias, enums para estados cerrados).

Motivadores que obligan a tomar una decisión sobre el motor de persistencia:

- **Motivador 1 — Modelo fuertemente relacional:** las entidades centrales (`clientes` , `polizas` , `siniestros` , `pagos` , `empresas_aseguradoras`) tienen relaciones 1:N reales y duras. Las consultas más frecuentes del panel requieren joins: listar pólizas de un cliente con su aseguradora y su próxima fecha de vencimiento; ver siniestros con todos sus documentos y la póliza asociada; reportes de pagos agrupados por cliente.
- **Motivador 2 — Consistencia transaccional (RNF Trazabilidad e Integridad, Entrega 1):** ciertas operaciones tocan varias tablas en un mismo flujo —dar de alta un siniestro crea el registro en `siniestros` y N registros en `siniestro_documentos` en una sola transacción; validar un pago actualiza `pagos` y propaga estado a las pólizas vinculadas. Si una falla, ninguna debe quedar a medias.
- **Motivador 3 — Trazabilidad por actor:** el modelo agrega `modificado_por_id` y `modificado_en` en cada tabla de negocio (decisión 1.6 del modelo de datos). Esto exige FKs con `SET NULL` , índices y constraints que un motor relacional resuelve nativamente.
- **Motivador 4 — Experiencia del equipo:** los 6 integrantes tienen experiencia con SQL y con Prisma (el proyecto ya nació con `prisma/schema.prisma`). Ninguno tiene experiencia productiva con NoSQL en sistemas con dominio relacional como este.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
MongoDB (NoSQL documental)	Schema flexible, ideal si las entidades fueran agregados autocontenidos. Buen rendimiento en lectura por documento.	El dominio es relacional, no documental : un cliente no contiene sus pólizas, pertenecen a una entidad distinta con su propio ciclo. Implementar transacciones multi-documento y joins en aplicación añade complejidad e impide expresar los constraints temporales (1.10 del modelo). Contradice motivadores 1 y 2.

MySQL	Ampliamente conocido, soporta transacciones ACID, buena comunidad.	Soporte más débil para tipos avanzados que el modelo usa o usará pronto (enum nativo limitado, JSONB, CHECK con expresiones, Decimal(15,2) con casts predecibles). Menos cómodo para evolucionar el schema con migraciones complejas. No aporta nada relevante por encima de la alternativa elegida en este caso.
PostgreSQL administrado por Supabase + Prisma ORM (elegida)	ACID completo, joins nativos, enums, decimales precisos, índices compuestos, CHECK constraints SQL, JSONB para documentos; Supabase elimina la operación del motor y aporta connection pooling listo.	— (esta fue la elegida)

3. Decisión tomada

Se decide: usar **PostgreSQL administrado por Supabase** como base de datos única del panel `my-app`, accedido mediante **Prisma ORM**. El schema final es el descrito en `Entregables/Entrega 3/modelo-datos.md` y materializado en `my-app/prisma/schema.prisma`.

Fundamentación:

- Resuelve el motivador 1 (modelo relacional):** PostgreSQL ejecuta nativamente todos los joins que el panel necesita para listados y reportes (clientes con sus pólizas vigentes, siniestros con su póliza y aseguradora, pagos con las pólizas agrupadas). No hay que reescribir esa lógica en código aplicativo como pasaría con un store documental.
- Resuelve los motivadores 2 y 3 (consistencia y trazabilidad):** las transacciones ACID permiten que crear un siniestro con sus N documentos sea atómico, y los CHECK constraints (`fecha_fin_vigencia >= fecha_inicio_vigencia` , `fecha_reporte >= fecha_ocurrencia`) protegen invariantes a nivel motor. Los campos `modificado_por_id` / `modificado_en` con FKs `SET NULL` se expresan naturalmente.
- Resuelve el motivador 4 (experiencia):** el equipo ya escribió `prisma/schema.prisma`, hizo el seed y las queries en `lib/data/*`. Cambiar de motor invalidaría todo ese trabajo y agregaría riesgo de migración en una entrega ya avanzada. Prisma además da migraciones declarativas y tipos generados que reducen errores en compile-time.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Los listados, KPIs y reportes del panel se implementan en SQL/Prisma directo, sin lógica adicional para emular joins.	Rigidez de schema: cualquier cambio del modelo exige una migración Prisma explícita (<code>prisma migrate dev</code>). Iterar sobre el modelo agrega fricción comparado con un store schemaless.
La consistencia transaccional la garantiza el motor: el flujo de carga de siniestro +	Vendor lock-in parcial con Supabase: se usan dos URLs (pooling y directa) y storage gestionado. Migrar a

documentos es atómico sin lógica de compensación.	otro proveedor de Postgres requeriría reconfigurar variables y revisar uso de extensiones.
Prisma genera tipos TypeScript a partir del schema, eliminando una clase entera de errores de runtime.	Escalabilidad horizontal limitada: Postgres escala mejor verticalmente. Si el volumen creciera mucho más allá del esperado para un productor de seguros mediano, se necesitaría sharding o réplicas de lectura.
Connection pooling de Supabase listo desde el día uno, alineado con el runtime serverless del panel (ver ADR-004).	El chatbot (ADR-001) no comparte esta base por estar en otro servicio; mantiene su propio store local. Hay duplicación parcial de modelo que el contrato REST (ADR-003) debe mantener consistente.